c0c25029dd / ProjExD_Group02

Code Pull requests Actions Projects Wiki Security and quality Insights Settings

ProjExD_Group02 / README.md

c0c2500901 ベース f4d239b · 2 hours ago

Preview Code Blame 57 lines (29 loc) · 2.3 KB Raw

SURVIVAL ARENA

実行環境の必要条件

python >= 3.10

pygame >= 2.1

ゲームの概要

画面中央のプレイヤー（青色の丸）を操作し，四方から次々と出現して追いかけてくる敵（赤色の丸）から逃げ続ける2D見下ろし型サバイバルゲーム。

制限時間120秒間生き残ることを目指す。

ゲームの遊び方

W / A / S / D キー：プレイヤーキャラクターの上下左右移動

R キー：ゲームオーバーまたはクリア画面でゲームをリスタート

ESC キー：ゲームを終了

ゲームの実装

共通基本機能

背景およびプレイヤー，敵キャラクターの描画

WASDキーによるプレイヤーの移動制御（斜め移動の速度正規化および画面外への移動制限）

画面外からの敵のランダム生成およびプレイヤー追尾処理

プレイヤーと敵の当たり判定（接触時のHP減少処理および無敵時間・点滅制御）

画面上部へのHP，生存時間（スコア），残り時間の表示

HPが0になった際のGAME OVER表示，120秒生存時のCLEAR表示，リスタートおよび終了機能

分担追加機能

ダッシュ機能（担当：メンバー1）：SHIFTキー押下時にプレイヤーの移動速度を一時的に上昇させる機能

爆弾機能（担当：メンバー2）：SPACEキー押下で画面内の敵を一括排除する機能

シールド機能（担当：メンバー3）：Eキー押下で一時的に敵からのダメージを無効化するバリアを展開する機能

アイテム機能（担当：メンバー4）：フィールド上に回復やスコア加算アイテムを出現・取得させる機能

特殊な敵の追加（担当：メンバー5）：速度や移動パターンの異なる新しい種類の敵を追加する機能

メモ

コードは main.py の単一ファイル構成とし，オブジェクト指向（Player, Enemy, Game クラス）で設計されている。

実行時の作業ディレクトリを固定するため，スクリプト先頭に os.chdir(os.path.dirname(os.path.abspath(**file**))) を記述している。

メンバーごとの追加機能実装を円滑に行えるよう，各クラスおよび関数に型ヒント（Type Hints）とdocstringを明記している。

ProjExD_Group02 / geme_base.py

c0c25029dd アイテム機能 d1c01dc · 5 minutes ago

Code Blame 369 lines (292 loc) · 13.2 KB Raw

```python
import os
import random
import sys
import math
import pygame

# ワーキングディレクトリを作業ファイルの場所に設定
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# 定数定義
SCREEN_WIDTH = 900
SCREEN_HEIGHT = 600
FPS = 60
TIME_LIMIT = 120  # 制限時間（秒）

# 色の定義（RGB）
COLOR_BG = (50, 160, 50)       # 緑色（フィールド）
COLOR_PLAYER = (0, 100, 255)   # 青色（プレイヤー）
COLOR_ENEMY = (255, 50, 50)    # 赤色（敵）
COLOR_WHITE = (255, 255, 255)  # 白色
COLOR_RED = (200, 0, 0)        # 強調用赤
COLOR_HEAL = (0, 255, 0)       # 回復アイテム
COLOR_ITEM_SCORE = (255, 255, 0)  # スコアアイテム


class Player:
    """プレイヤーキャラクターの移動、HP、描画を管理するクラス"""

    def __init__(self, x: float, y: float) -> None:
        """プレイヤーの初期位置やステータスを設定する"""
        self.x: float = x
        self.y: float = y
        self.radius: int = 15
        self.speed: float = 4.0
        self.hp: int = 100
        self.max_hp: int = 100
        self.invincible_timer: int = 0  # 被弾後の無敵時間フレームカウンタ

    def update(self, keys: pygame.key.ScancodeWrapper) -> None:
        """キー入力を受けて移動処理を行い、画面外への逸脱を防ぐ"""
        dx: float = 0.0
        dy: float = 0.0

        # WASDキーによる入力判定
```

```python
        if keys[pygame.K_w]:
            dy -= 1.0
        if keys[pygame.K_s]:
            dy += 1.0
        if keys[pygame.K_a]:
            dx -= 1.0
        if keys[pygame.K_d]:
            dx += 1.0

        # 斜め移動時の速度が速くならないように正規化（1 / √2 ≒ 0.7071）
        if dx != 0.0 and dy != 0.0:
            dx *= 0.7071
            dy *= 0.7071

        # 位置の更新
        self.x += dx * self.speed
        self.y += dy * self.speed

        # 画面外に出ないように制限
        if self.x - self.radius < 0:
            self.x = self.radius
        elif self.x + self.radius > SCREEN_WIDTH:
            self.x = SCREEN_WIDTH - self.radius

        if self.y - self.radius < 0:
            self.y = self.radius
        elif self.y + self.radius > SCREEN_HEIGHT:
            self.y = SCREEN_HEIGHT - self.radius

        # 無敵タイマーのカウントダウン
        if self.invincible_timer > 0:
            self.invincible_timer -= 1

    def draw(self, screen: pygame.Surface) -> None:
        """プレイヤーを描画する（無敵時間中は点滅表現を行う）"""
        # 無敵時間中は点滅させる（5フレームごとに表示・非表示を切り替え）
        if self.invincible_timer > 0 and (self.invincible_timer // 5) % 2 == 0:
            return

        pygame.draw.circle(screen, COLOR_PLAYER, (int(self.x), int(self.y)), self.radius)
        pygame.draw.circle(screen, COLOR_WHITE, (int(self.x), int(self.y)), self.radius, 2)

    def get_pos(self) -> tuple[float, float]:
        """プレイヤーの現在座標を返す"""
        return (self.x, self.y)

    def get_hp(self) -> int:
        """プレイヤーの現在のHPを返す"""
        return self.hp

    def take_damage(self, amount: int) -> None:
        """ダメージを受け、無敵時間を付与する"""
        if self.invincible_timer == 0:
            self.hp -= amount
            if self.hp < 0:
                self.hp = 0
            self.invincible_timer = 60  # 1秒間（60フレーム）の無敵時間

```

```
103         def heal(self, amount: int) -> None:
104             """HPを回復する（最大HPを超えない）"""
105             self.hp = min(self.max_hp, self.hp + amount)
106 
107 
108     class Enemy:
109         """敵キャラクターの移動と描画を管理するクラス"""
110 
111         def __init__(self, x: float, y: float) -> None:
112             """敵の初期位置を設定する"""
113             self.x: float = x
114             self.y: float = y
115             self.radius: int = 12
116             self.speed: float = 2.2
117 
118         def update(self, player_x: float, player_y: float) -> None:
119             """プレイヤーの方向に向かって移動する"""
120             dx: float = player_x - self.x
121             dy: float = player_y - self.y
122             distance: float = math.hypot(dx, dy)
123 
124             # プレイヤーに向かって単位ベクトルを掛けて移動
125             if distance > 0:
126                 dx /= distance
127                 dy /= distance
128                 self.x += dx * self.speed
129                 self.y += dy * self.speed
130 
131         def draw(self, screen: pygame.Surface) -> None:
132             """敵を描画する"""
133             pygame.draw.circle(screen, COLOR_ENEMY, (int(self.x), int(self.y)), self.radius)
134             pygame.draw.circle(screen, COLOR_WHITE, (int(self.x), int(self.y)), self.radius, 2)
135 
136         def get_pos(self) -> tuple[float, float]:
137             """敵の現在座標を返す"""
138             return (self.x, self.y)
139 
140 
141     class Item:
142         """アイテムの種類、位置、描画を管理するクラス"""
143 
144         def __init__(self, x: float, y: float, item_type: str) -> None:
145             self.x: float = x
146             self.y: float = y
147             self.radius: int = 10
148             self.item_type: str = item_type
149 
150         def draw(self, screen: pygame.Surface) -> None:
151             """アイテムを種類に応じた色で描画する"""
152             if self.item_type == "HEAL":
153                 color = COLOR_HEAL
154             else:
155                 color = COLOR_ITEM_SCORE
156 
157             pygame.draw.circle(
158                 screen, color,
159                 (int(self.x), int(self.y)), self.radius
160             )
```

```python
    def get_pos(self) -> tuple[float, float]:
        """アイテムの座標を返す"""
        return (self.x, self.y)


class Game:
    """ゲーム全体の状態、時間、スコア、当たり判定を管理するクラス"""

    def __init__(self) -> None:
        """Pygameとゲーム変数を初期化する"""
        pygame.init()
        self.screen: pygame.Surface = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
        pygame.display.set_caption("SURVIVAL ARENA")
        self.clock: pygame.time.Clock = pygame.time.Clock()
        self.font: pygame.font.Font = pygame.font.SysFont(None, 36)
        self.large_font: pygame.font.Font = pygame.font.SysFont(None, 64)

        self.reset_game()

    def reset_game(self) -> None:
        """ゲームの状態を初期化またはリスタートする"""
        self.player: Player = Player(float(SCREEN_WIDTH / 2), float(SCREEN_HEIGHT / 2))
        self.enemies: list[Enemy] = []
        self.items: list[Item] = []
        self.item_spawn_timer: int = 0
        self.item_score: int = 0
        self.elapsed_frames: int = 0
        self.spawn_timer: int = 0
        self.game_state: str = "PLAYING"  # "PLAYING", "GAME_OVER", "CLEAR"

    def run(self) -> None:
        """メインゲームループを実行する"""
        running: bool = True
        while running:
            # イベント処理
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False
                elif event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_ESCAPE:
                        running = False
                    elif event.key == pygame.K_r:
                        # ゲームオーバーまたはクリア時にRキーでリスタート
                        if self.game_state in ["GAME_OVER", "CLEAR"]:
                            self.reset_game()

            keys = pygame.key.get_pressed()

            if self.game_state == "PLAYING":
                # プレイヤーの更新
                self.player.update(keys)

                # アイテムの出現処理（約10秒ごと）
                self.item_spawn_timer += 1
                if self.item_spawn_timer >= 600:
                    self.item_spawn_timer = 0
                    self.spawn_item()
```

```
                # 時間経過のカウント
                self.elapsed_frames += 1
                survived_seconds = self.elapsed_frames // FPS
                if survived_seconds >= TIME_LIMIT:
                    self.game_state = "CLEAR"

                # 一定時間ごと（約1.5秒おごと）に敵を出現させる
                self.spawn_timer += 1
                if self.spawn_timer >= 90:
                    self.spawn_timer = 0
                    self.spawn_enemy()

                # 敵の更新
                px, py = self.player.get_pos()
                for enemy in self.enemies:
                    enemy.update(px, py)

                # アイテムの取得判定
                self.check_item_collisions()

                # 当たり判定のチェック
                self.check_collisions()

                # HPが0になったらゲームオーバー
                if self.player.get_hp() <= 0:
                    self.game_state = "GAME_OVER"

            # 描画処理
            self.draw()

            self.clock.tick(FPS)

        pygame.quit()
        sys.exit()

    def spawn_enemy(self) -> None:
        """画面の上下左右の端からランダムに敵を生成する"""
        side = random.randint(0, 3)
        if side == 0:  # 上端
            x = random.randint(0, SCREEN_WIDTH)
            y = -20
        elif side == 1:  # 下端
            x = random.randint(0, SCREEN_WIDTH)
            y = SCREEN_HEIGHT + 20
        elif side == 2:  # 左端
            x = -20
            y = random.randint(0, SCREEN_HEIGHT)
        else:  # 右端
            x = SCREEN_WIDTH + 20
            y = random.randint(0, SCREEN_HEIGHT)

        self.enemies.append(Enemy(float(x), float(y)))

    def spawn_item(self) -> None:
        """フィールド上にランダムなアイテムを生成する"""
        x = random.randint(30, SCREEN_WIDTH - 30)
        y = random.randint(50, SCREEN_HEIGHT - 30)

```

```
278             item_type = random.choice(["HEAL", "SCORE"])
279             self.items.append(Item(float(x), float(y), item_type))
280 
281         def check_item_collisions(self) -> None:
282             """プレイヤーとアイテムの接触判定・取得処理"""
283             px, py = self.player.get_pos()
284 
285             for item in self.items[:]:
286                 ix, iy = item.get_pos()
287                 distance = math.hypot(px - ix, py - iy)
288 
289                 if distance < self.player.radius + item.radius:
290                     if item.item_type == "HEAL":
291                         self.player.heal(30)
292                     elif item.item_type == "SCORE":
293                         self.item_score += 100
294 
295                     self.items.remove(item)
296 
297         def check_collisions(self) -> None:
298             """プレイヤーとすべての敵の接触判定を行う"""
299             px, py = self.player.get_pos()
300             p_radius = self.player.radius
301 
302             for enemy in self.enemies:
303                 ex, ey = enemy.get_pos()
304                 e_radius = enemy.radius
305                 distance = math.hypot(px - ex, py - ey)
306 
307                 # 円同士の当たり判定
308                 if distance < p_radius + e_radius:
309                     self.player.take_damage(20)  # 接触時に20ダメージ
310 
311         def draw(self) -> None:
312             """画面全体の描画を行う"""
313             # 背景描画
314             self.screen.fill(COLOR_BG)
315 
316             # キャラクター描画
317             self.player.draw(self.screen)
318             for enemy in self.enemies:
319                 enemy.draw(self.screen)
320 
321             # アイテム描画
322             for item in self.items:
323                 item.draw(self.screen)
324 
325             # UI（HP、スコア、残り時間）の描画
326             hp_text = self.font.render(f"HP: {self.player.get_hp()}/{self.player.max_hp}", True, COLOR_WHITE)
327             self.screen.blit(hp_text, (20, 20))
328 
329             survived_seconds = self.elapsed_frames // FPS
330             total_score = survived_seconds + self.item_score
331 
332             score_text = self.font.render(
333                 f"Score: {total_score}", True, COLOR_WHITE
334             )
335             self.screen.blit(score_text, (330, 20))
```

```
336
337             remaining_seconds = max(0, TIME_LIMIT - survived_seconds)
338             time_text = self.font.render(f"Limit: {remaining_seconds}s", True, COLOR_WHITE)
339             self.screen.blit(time_text, (730, 20))
340
341             # ゲーム終了時のオーバーレイとメッセージ
342             if self.game_state == "GAME_OVER":
343                 overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT), pygame.SRCALPHA)
344                 overlay.fill((0, 0, 0, 160))
345                 self.screen.blit(overlay, (0, 0))
346
347                 game_over_text = self.large_font.render("GAME OVER", True, COLOR_RED)
348                 restart_text = self.font.render("Press R to Restart | Press ESC to Quit", True, COLOR_WHITE)
349
350                 self.screen.blit(game_over_text, (SCREEN_WIDTH // 2 - game_over_text.get_width() // 2, SCREEN_H
351                 self.screen.blit(restart_text, (SCREEN_WIDTH // 2 - restart_text.get_width() // 2, SCREEN_HEIGH
352
353             elif self.game_state == "CLEAR":
354                 overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT), pygame.SRCALPHA)
355                 overlay.fill((0, 0, 0, 160))
356                 self.screen.blit(overlay, (0, 0))
357
358                 clear_text = self.large_font.render("CLEAR!", True, (255, 255, 0))
359                 restart_text = self.font.render("Press R to Restart | Press ESC to Quit", True, COLOR_WHITE)
360
361                 self.screen.blit(clear_text, (SCREEN_WIDTH // 2 - clear_text.get_width() // 2, SCREEN_HEIGHT //
362                 self.screen.blit(restart_text, (SCREEN_WIDTH // 2 - restart_text.get_width() // 2, SCREEN_HEIGH
363
364             pygame.display.flip()
365
366
367     if __name__ == "__main__":
368         game = Game()
369         game.run()
```

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: c0c25029/アイテム機能

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests **Create pull request**

1 commit | 1 file changed | 1 contributor

Commits on Nov 3, 2026

アイテム機能

c0c25029dd committed 11 minutes ago d1c01dc

Showing 1 changed file with **77 additions** and **1 deletion**. Split Unified

78 geme_base.py

```
19  19    COLOR_ENEMY = (255, 50, 50)    # 赤色（敵）
20  20    COLOR_WHITE = (255, 255, 255)  # 白色
21  21    COLOR_RED = (200, 0, 0)        # 強調用赤
    22  + COLOR_HEAL = (0, 255, 0)       # 回復アイテム
    23  + COLOR_ITEM_SCORE = (255, 255, 0)  # スコアアイテム
22  24
23  25
24  26    class Player:
98  100                   self.hp = 0
99  101               self.invincible_timer = 60  # 1秒間（60フレーム）の無敵時間
100 102
    103 +     def heal(self, amount: int) -> None:
    104 +         """HPを回復する（最大HPを超えない）"""
    105 +         self.hp = min(self.max_hp, self.hp + amount)
    106 +
101 107
102 108   class Enemy:
103 109       """敵キャラクターの移動と描画を管理するクラス"""
132 138           return (self.x, self.y)
133 139
134 140
    141 + class Item:
    142 +     """アイテムの種類、位置、描画を管理するクラス"""
    143 +
```

```
+    def __init__(self, x: float, y: float, item_type: str) -> None:
+        self.x: float = x
+        self.y: float = y
+        self.radius: int = 10
+        self.item_type: str = item_type
+
+    def draw(self, screen: pygame.Surface) -> None:
+        """アイテムを種類に応じた色で描画する"""
+        if self.item_type == "HEAL":
+            color = COLOR_HEAL
+        else:
+            color = COLOR_ITEM_SCORE
+
+        pygame.draw.circle(
+            screen, color,
+            (int(self.x), int(self.y)), self.radius
+        )
+
+    def get_pos(self) -> tuple[float, float]:
+        """アイテムの座標を返す"""
+        return (self.x, self.y)
+
+
class Game:
    """ゲーム全体の状態、時間、スコア、当たり判定を管理するクラス"""

        """ゲームの状態を初期化またはリスタートする"""
        self.player: Player = Player(float(SCREEN_WIDTH / 2), float(SCREEN_HEIGHT / 2))
        self.enemies: list[Enemy] = []
+       self.items: list[Item] = []
+       self.item_spawn_timer: int = 0
+       self.item_score: int = 0
        self.elapsed_frames: int = 0
        self.spawn_timer: int = 0
        self.game_state: str = "PLAYING"  # "PLAYING", "GAME_OVER", "CLEAR"
                # プレイヤーの更新
                self.player.update(keys)

+               # アイテムの出現処理（約10秒ごと）
+               self.item_spawn_timer += 1
+               if self.item_spawn_timer >= 600:
+                   self.item_spawn_timer = 0
+                   self.spawn_item()
+
                # 時間経過のカウント
                self.elapsed_frames += 1
                survived_seconds = self.elapsed_frames // FPS
                for enemy in self.enemies:
                    enemy.update(px, py)

+               # アイテムの取得判定
+               self.check_item_collisions()
+
                # 当たり判定のチェック
                self.check_collisions()
```

```
227 271             self.enemies.append(Enemy(float(x), float(y)))
228 272
    273 +     def spawn_item(self) -> None:
    274 +         """フィールド上にランダムなアイテムを生成する"""
    275 +         x = random.randint(30, SCREEN_WIDTH - 30)
    276 +         y = random.randint(50, SCREEN_HEIGHT - 30)
    277 +
    278 +         item_type = random.choice(["HEAL", "SCORE"])
    279 +         self.items.append(Item(float(x), float(y), item_type))
    280 +
    281 +     def check_item_collisions(self) -> None:
    282 +         """プレイヤーとアイテムの接触判定・取得処理"""
    283 +         px, py = self.player.get_pos()
    284 +
    285 +         for item in self.items[:]:
    286 +             ix, iy = item.get_pos()
    287 +             distance = math.hypot(px - ix, py - iy)
    288 +
    289 +             if distance < self.player.radius + item.radius:
    290 +                 if item.item_type == "HEAL":
    291 +                     self.player.heal(30)
    292 +                 elif item.item_type == "SCORE":
    293 +                     self.item_score += 100
    294 +
    295 +                 self.items.remove(item)
    296 +
229 297         def check_collisions(self) -> None:
230 298             """プレイヤーとすべての敵の接触判定を行う"""
231 299             px, py = self.player.get_pos()
250 318             for enemy in self.enemies:
251 319                 enemy.draw(self.screen)
252 320
    321 +         # アイテム描画
    322 +         for item in self.items:
    323 +             item.draw(self.screen)
    324 +
253 325             # UI（HP、スコア、残り時間）の描画
254 326             hp_text = self.font.render(f"HP: {self.player.get_hp()}/{self.player.max_hp}",
    True, COLOR_WHITE)
255 327             self.screen.blit(hp_text, (20, 20))
256 328
257 329             survived_seconds = self.elapsed_frames // FPS
258     -           score_text = self.font.render(f"Score (Time): {survived_seconds}s", True,
    COLOR_WHITE)
    330 +         total_score = survived_seconds + self.item_score
    331 +
    332 +         score_text = self.font.render(
    333 +             f"Score: {total_score}", True, COLOR_WHITE
    334 +         )
259 335             self.screen.blit(score_text, (330, 20))
260 336
261 337             remaining_seconds = max(0, TIME_LIMIT - survived_seconds)
```